

目录

消息队列Mafka

🔍 | 📄 正文 6789 字 (10.3 页) | 浏览 9 次 (1 人) | 全部 >

🔗 AI 速览 点击生成简要速览, 支持更多 风格切换 ▾ 助你高效阅读

简介

mafka是一个高可用、可拓展、高性能的分布式消息队列服务, 广泛用于消息异步处理、应用解耦、流量削峰、发布 / 订阅模型等场景

Mafka底层基于Apache Kafka, 增加了自研的基于机房粒度的中心化调度、时间回溯、粘性分配、死信、延迟队列、适用于美团自用的同步/异步客户端、机房容灾等高阶特性。

典型的适用场景有:

1. 流量削峰, 如秒杀、抢红包等。
2. 异步处理模式, 请求方将请求发给 Mafka 之后, 即可返回, 不需要等处理方响应, 减少无效等待时间。
3. 应用系统之间的解耦合, 发送者和接受者不需要受对方限制、只需要确认消息, 比如发送和接收者可以是不同的系统, 不同的语言编写的, 地理上可以不在同一个地域, 甚至不需同时在线。
4. 发布订阅 (Pub/Sub), 一条消息支持广播给多个消费者。

队列类型

1. 普通队列: 这种就是最基础的普通消息队列, 支持消息生产和消费。
2. 延迟队列: 指将生产者发送的消息, 延时一定的时间后再传给消费者进行消费。如超过 10 分钟未支付, 订单自动取消场景。

消息存储可靠性

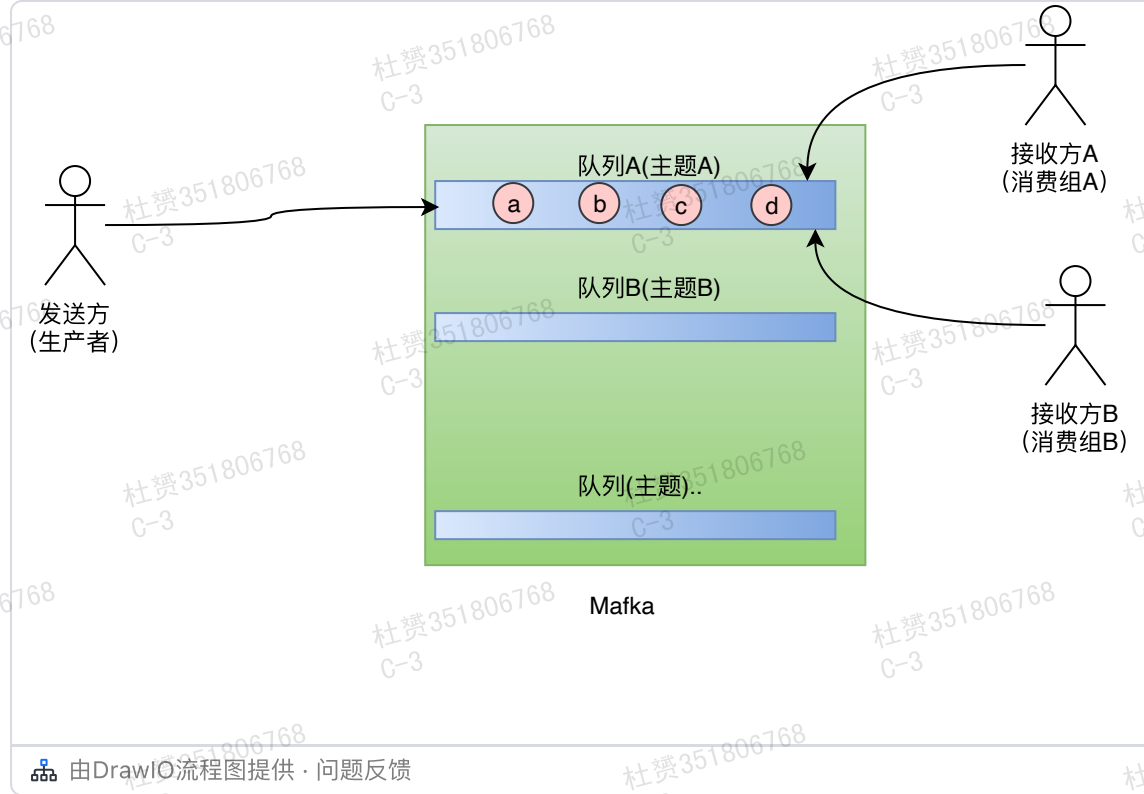
Mafka 通过多副本策略保障消息存储的可靠性, 单副本宕机不会影响整个集群的消息转发, 并且具备副本数量自定义的能力, 用户可以通过增加副本数量 (会带来成本的增加) 进一步提升可靠性。

消息通讯可靠性

通过 ACK (Acknowledge, 简写为 ACK, 告知已收到) 机制保障可靠性。一条消息, 必须有 2 台或 3 台机器接收到后才算发送成功。

衡量指标	目标	说明
可用性	服务可用性不低于 99.995%	服务可用性是服务处于可用状态的时间与总时间的比值，一般以年为单位来计算。美团根据自身服务的特点，给出了可用性计算方法，详细参考： 可用性计算方法
消息可靠性	可靠性超过 99.995%	可靠性是指消息丢失数量对消息总数量的占比，以月为单位来计算。
性能	99% 发送延时控制在 100ms 99.9% 消息进出延时控制在 100ms	发送延是用户发送消息需要的时间，消息从用户发出到 Mafka 返回生产成功； 进出延时是指消息从进入 Mafka，到用户能在消费端看到这条消息。 * 具体性能指标取决于集群机器配置和实际集群状态，参见 * Mafka 性能测试 试。 * 消息进出延迟主要取决于业务侧实际需要和消费速率，不作为 SLA 承诺
功能支持	满足业务方新需求或潜在需求	1. 优先级队列、事务消息 2. 支持业务流量隔离 (泳道、SET/LiteSET) 3. 支持死信、消息回溯

业务模型



mafkabased on kafka做了哪些扩展?

1. MQ feature建设

- 实现了死信队列的支持
- 实现了按时间方式的消息回溯和前进(2016年当时Kafka官方还不支持时间回溯, Mafka当时是自研的这个功能)
- 消息粒度的延迟, 按自定义时间的方式, 发定时消息, 支持5s到30天内的任意自定义时间。
- 队列粒度的消费延迟, 支持定义时间延迟消费整个队列。
- 生产、消费支持并行生产、并行消费, 提高生产和消费吞吐量。
- 事务性消息, 两阶段提交方式实现的事务性消息。
- Push方式消费, 消费者通过RPC消费消息, 解决了消费者数量和分区数量的耦合。
- 消息轨迹, 消息查找, 支持以定义方式查询。

- i. 消息批量消费功能
- j. 消费异常重试次数自定义，或跳过有问题消息
- k. 消费端在消费消息时，消费者上下线，支持粘性分配，减少分区变动

2. 适用于美团公司业务feature建设

- a. 基于环境隔离的消息生产和消费
- b. 支持故障注入，为客户端提供故障模拟功能，用于线上事故演练等场景。
- c. 消息生产和消费以set方式隔离
- d. 灰度链路，支持业务线上灰度链路发布。
- e. 全链路压测支持，方便业务在线上做全链路压测
- f. 主题鉴权，方便业务对队列做生产和消费权限控制
- g. Storm/Flink方式接入生产和消费

3. 中心化多集群调度能力建设

原生的Kafka是没有机房和BG概念的，只是一个简单的集群。Mafka为了适应美团多机房、多BG业务特性，增加了按BG、机房、地域等维度的调度多集群生产和消费调度功能。

- a. 同/跨机房生产和消费调度
- b. 同/跨地域生产和消费调度
- c. 多集群同时生产和消费
- d. 自定义分组生产和调度，方便跨云用户实现个性化的调度策略。

4. 高可用、高可靠、高并发能力建设

- a. 基于分片、集群、或机房粒度的容灾策略，在某个分片、集群、机房出现故障和不可用时，实现了RTO<60s，RPO≈1s的容灾能力，仍然保持整个队列高可用，Mafka提供99.999%的SLA可用性保障。

- b. 对kafka依赖zookeeper的改造，从临时结点改造为持久节点，引入第三方scanner服务来做broker健康检查，实现秒级的节点状态改变，减少由于Broker机器负载、网络抖动等因素带来的集群抖动。
- c. 自研的同步和异步客户端，让消息的生产和消费实现靠可靠，不漏消息，不丢消息，尽最大可能不重复消费消息。
- d. 自研的生产客户端能同时向同机房、同地域、全地域多个集群生产消息，实现集群容量的跨机房、跨地域横向水平扩容。内置的熔断保护器，自动探测、熔断问题集群，实现秒级主动容灾功能，保持队列的高可用。
- e. 完全自研的集群控制器，能实现最优的集群内队列分片分布，达到集群内负载均衡，并能实现跨机房集群队列分片平均分布，实现更高可靠性的容灾策略。
- f. 360度的队列监控，保障用户在第一时间收到队列的运营数据变化，包括消息消费积压，消息生产、和消费速度变化，生产端、消费端数量的变化。
- g. 参考官方新版本的性能优化方案，自研了消息拉取和复制的时延优化方案。

5. 多语言客户端建设

- a. Mafka官方支持java/c++/php三种语言的客户端
- b. 此外，Mafka公布了客户端协议，并协助业务一起共建的客户端包括: Node.js/Python/Go等。

6. 用户平台、运维平台建设

- a. 完全自研的用户平台，以用户为中心，方便业务侧队列、消费组申请和审批，配置管理。图形化监控队列信息，消息信息，队列积压信息等。日常运维需求，用户可以完全自助完成审批，审批，报警自定义配置。
- b. 完全自研的运维平台，集中管理集群，创建、修改、扩缩容等日常操作，实现“一键式”自动化运维操作，方便SRE管理集群。大盘式集群监控，实时展现集群状态。为SRE衡量集群稳定性指标做直观的展示。

Mafka消息队列详细介绍

一、Mafka产品概述

Mafka是美团自研的分布式消息队列中间件，基于Apache Kafka进行深度改造和优化。自2016年开始研发，目前已发展到2.0版本，是美团点评业务场景中不可或缺的基础设施组件。

核心特性

- **高可用性**：服务可用性达到99.995%以上
- **高性能**：支持万亿级日调用量，99%发送延时控制在100ms内
- **高扩展性**：支持水平扩展，适应业务快速增长
- **全托管服务**：用户只需专注业务开发，无需部署运维

技术架构

Mafka主要包含三个核心组件：

1. **Broker**：消息存储节点，多个Broker组成集群
2. **Castle**：中控调度服务，负责客户端路由调度
3. **ClientSDK**：客户端SDK，支持Java、C++、PHP、Go、Python、Node.js等多种语言

二、Mafka核心功能

1. 基础消息功能

- **普通消息**：标准的消息发送和消费
- **批量消息**：支持批量发送和消费，提高吞吐量
- **事务消息**：保证消息发送的原子性
- **优先级消息**：支持消息优先级处理

2. 高级特性

- **延迟消息/延迟队列**：支持消息延迟投递

• **Push消费**：打破分区数量限制，支持更多消费者

- **死信队列**：处理消费失败的消息
- **消息回溯**：支持最大7天的消息重复消费
- **流量录制与回放**：支持消息轨迹追踪

3. 容灾与调度

- **中心化调度**：基于机房粒度的智能调度
- **多机房容灾**：支持跨AZ生产和消费
- **同机房优先**：优化网络延迟
- **自动故障转移**：单机故障30s内恢复，机房故障1-5分钟内恢复

4. 治理能力

- **环境隔离**：支持泳道1.0和泳道2.0
- **Set化支持**：支持业务隔离
- **监控告警**：完善的监控体系和告警机制
- **权限管理**：支持主题和消费组级别的鉴权

三、Mafka面试题目汇总（按重要程度排序）

🔥 核心重点题目（必考）

1. Mafka的整体架构是怎样的？各组件的作用是什么？

详细回答：

Mafka采用分布式架构，主要包含三个核心组件：

Broker集群：

- 负责消息的实际存储和处理

• 多个Broker组成集群，提供高可用性

- 每个Broker管理多个Topic的分区
- 支持消息的持久化存储，默认保存7天

Castle调度中心：

- 中控调度服务，负责客户端的路由调度
- 决定客户端应该连接哪个Broker进行生产或消费
- 实现同机房优先、同地域优先等调度策略
- 支持故障转移和负载均衡

ClientSDK：

- 提供多语言客户端SDK（Java、C++、Go、Python等）
- 与Castle保持心跳，接收调度指令
- 实现消息的发送、消费、重试等逻辑
- 支持同步、异步、批量等多种调用方式

工作流程：

1. 客户端启动后向Castle发送心跳请求
2. Castle返回应该连接的Broker信息
3. 客户端连接指定Broker进行消息生产或消费
4. 持续与Castle保持心跳，接收调度变更

2. Mafka如何保证消息的可靠性？ACK机制是怎样的？

详细回答：

Mafka通过多层机制保证消息可靠性：

ACK确认机制：

- **ACK=0：**生产者发送消息后不等待确认，性能最高但可能丢消息

• **ACK=1**：等待主副本确认，平衡性能和可靠性，极端情况可能丢失1秒内消息

- **ACK=-1**：等待所有副本确认，可靠性最高但延迟较大

多副本机制：

- 默认2-3个副本，容忍1-2台机器同时宕机
- 主副本负责读写，从副本负责备份
- 副本间通过同步线程保持数据一致性

消费端可靠性：

- 支持三种消费状态：`CONSUME_SUCCESS`、`RECONSUME_LATER`、`CONSUMER_FAILURE`
- 消费失败支持重试机制，最多重试3次
- 支持死信队列处理无法消费的消息

存储可靠性：

- 消息持久化到磁盘
- 支持数据压缩和清理策略
- 提供消息轨迹追踪能力

3. Mafka的分区机制是如何工作的？如何保证消息顺序性？

详细回答：

分区机制：

- 每个Topic可以有多个分区 (Partition)
- 消息根据Key进行分区路由，相同Key的消息进入同一分区
- 分区数量影响消费者并发度，消费者数量不能超过分区数量
- 支持动态调整分区数量（需要联系Mafka团队）

消息顺序性保证：

- **分区内有序**：同一分区内的消息严格按照发送顺序存储和消费

● **全局无序**：不同分区间的消息无法保证顺序

● **顺序消费配置**：

- 生产端：使用相同的Key确保消息进入同一分区
- 消费端：设置 `consumer.parallel.num=1` 确保单线程消费

顺序性最佳实践：

- 需要严格顺序的消息使用相同的分区Key
- 避免消费端并行处理破坏顺序性
- 考虑使用单分区Topic（影响并发性能）

4. Mafka如何实现高可用性？容灾机制是怎样的？

详细回答：

多层容灾机制：

单机故障容灾：

- $RTO \leq 30$ 秒， $RPO = 0$ 秒
- 多副本机制，主副本故障时自动切换到从副本
- 客户端30秒内自动转移到其他健康节点

机房故障容灾：

- $RTO \leq 1$ 分钟（同步） / ≤ 5 分钟（异步）
- 支持跨AZ部署和自动切换
- Castle调度中心实现智能路由切换
- 客户端版本 $\geq 3.6.9$ 支持自动容灾

调度策略：

- **同机房优先**：优先使用同机房资源，减少网络延迟
- **同地域优先**：机房故障时切换到同城其他机房

- **全部集群：**极端情况下使用所有可用集群

数据可靠性保障：

- 机房断网场景：数据保存至孤岛服务器，RPO=0
- 机房断电场景：增量数据RPO=0，存量数据根据副本情况决定

监控与告警：

- 实时监控集群健康状态
- 自动故障检测和切换
- 完善的告警通知机制

5. Mafka的消费模型有哪些？Push和Pull的区别是什么？

详细回答：

消费模型类型：

Pull消费（默认）：

- 消费者主动从Broker拉取消息
- 消费者数量受分区数量限制
- 支持批量拉取，提高效率
- 消费者可以控制消费速度

Push消费：

- 通过PushServer推送消息给消费者
- 打破分区数量限制，支持更多消费者
- PushServer作为代理层，从Topic消费后推送给业务消费者
- 适用于需要大量消费者的场景

两种模式对比：

特性	Pull消费	Push消费
消费者数量限制	受分区数量限制	不受分区数量限制
消息顺序性	分区内有序	无法保证分区内有序
消息可靠性	高可靠性	极端场景可能丢消息
性能	高性能	相对较低
适用场景	常规业务场景	需要大量消费者的场景

Push消费配置：

- 消费组类型设置为Push类型
- 配置Push消费线程数（默认1，最大32）
- 设置Push超时时间（默认100s）

重要题目

6. Mafka的延迟消息是如何实现的？有什么限制？

详细回答：

延迟消息实现原理：

- 使用DelayServer服务暂存延迟消息
- 消息先发送到DelayServer，等待指定时间后再投递给消费者
- 支持自定义延迟时间（5s、10s、1分钟、1小时、30天等）

两种延迟类型：

- **延迟队列：**整个队列的所有消息都延迟相同时间
- **延迟消息：**队列中每条消息可以设置不同的延迟时间

使用限制：

• 小于5s的延迟消息需要特殊申请

- 延迟消息QPS建议小于5000
- 延迟时间最长支持30天
- 延迟消息会占用额外的存储资源

应用场景：

- 订单超时自动取消
- 定时任务触发
- 业务流程延迟处理

7. Mafka如何处理消息积压问题？

详细回答：

积压产生原因：

- 消费者处理能力不足
- 消费者实例数量不够
- 消费逻辑存在性能瓶颈
- 消费者实例故障或重启

解决方案：

1. 增加消费者实例：

- 增加消费者机器数量
- 确保消费者数量不超过分区数量

2. 优化消费性能：

- 调整 `consumer.parallel.num` 增加单分区并行消费线程
- 使用批量消费接口提高处理效率
- 优化消费逻辑，减少处理时间

3. 使用Push消费：

- 打破分区数量限制
- 支持更多消费者并行处理

4. 临时扩容分区：

- 联系Mafka团队增加分区数量
- 重新分配消费者到新分区

监控指标：

- 消费延迟（Lag）
- 消费速率（QPS）
- 消费者实例状态

8. Mafka的事务消息是如何实现的？

详细回答：

事务消息原理：

- 通过消息标签机制实现（未提交、已提交）
- 未提交的消息不会被消费者看到
- 支持生产端事务，可以将入库和发消息封装为一个事务

事务流程：

1. 开始事务
2. 发送消息（标记为未提交状态）
3. 执行业务逻辑（如数据库操作）
4. 提交事务（消息变为已提交状态，消费者可见）
5. 回滚事务（消息被删除，消费者不可见）

使用限制：

• QPS小于5000

- 一个主题只能有一个事务发送方
- 需要特殊申请开启事务功能

适用场景:

- 需要保证消息发送和数据库操作原子性的场景
- 分布式事务处理
- 关键业务流程的一致性保证

9. Mafka的泳道机制是如何工作的？ 1.0和2.0有什么区别？

详细回答:

泳道1.0机制:

- 为每个泳道创建独立的Topic和消费组
- 不同泳道的消息完全隔离
- 泳道消息只能被对应泳道的消费者消费
- 如果泳道Topic不存在，消息发往骨干环境

泳道2.0机制:

- 摒弃泳道Topic概念，只创建泳道消费组
- 所有消息都发往骨干Topic
- 通过MTrace Context在消息上打泳道标识
- 消费者根据泳道标识选择性消费消息

两个版本对比:

特性	泳道1.0	泳道2.0
资源消耗	高（需要创建多个Topic）	低（共享骨干Topic）

调度灵活性	低（泳道消息到骨干后无法被泳道消费）	高（支持灵活调度）
实现复杂度	简单	相对复杂
兼容性	向后兼容	需要客户端支持

泳道2.0优势：

- 节约Mafka资源
- 支持泳道消息被骨干消费（当泳道消费者不存在时）
- 更灵活的流量调度策略

🔥 中等重要题目

10. Mafka如何进行性能优化？

详细回答：

生产端优化：

- 使用异步发送提高吞吐量
- 配置连接池大小（特别是多线程场景）
- 调整ACK级别平衡性能和可靠性
- 使用批量发送减少网络开销
- 启用消息压缩（gzip、lz4、snappy）

消费端优化：

- 调整 `consumer.parallel.num` 增加并行度
- 使用批量消费接口
- 优化消费逻辑，减少处理时间
- 合理设置消费者实例数量

网络优化：



• 使用同机房优先策略减少网络延迟

- 配置合适的网络参数
- 监控网络质量和延迟

存储优化:

- 合理设置消息保存时间
- 配置消息压缩策略
- 定期清理过期数据

11. Mafka的监控体系是怎样的?

详细回答:

监控维度:

- **服务可用性:** 集群、Topic、消费组级别的可用性监控
- **性能指标:** QPS、延迟、吞吐量等
- **资源使用:** CPU、内存、磁盘、网络等
- **业务指标:** 消息积压、消费延迟等

监控工具:

- **管理平台:** 提供可视化监控界面
- **Raptor:** 性能指标监控和告警
- **CAT:** 调用链追踪和性能分析
- **MTrace:** 分布式链路追踪

告警机制:

- 支持多种告警方式 (邮件、短信、太象群)
- 可配置告警阈值和策略
- 支持告警收敛和升级

关键指标：

- 消息生产成功率
- 消息消费成功率
- 消费延迟（Lag）
- 集群健康状态

12. Mafka如何支持大消息处理？

详细回答：

消息大小限制：

- 默认支持1MB以下消息
- 特殊需求可申请5MB大消息支持
- 超大消息建议进行压缩处理

大消息处理策略：

- **消息压缩**：使用gzip、lz4、snappy等压缩算法
- **消息分片**：将大消息拆分为多个小消息
- **外部存储**：将大数据存储到外部系统，消息中只传递引用

配置调整：

- 调整 `max.message.bytes` 参数
- 配置 `fetch.message.max.bytes` 拉取参数
- 调整网络缓冲区大小

注意事项：

- 大消息会影响网络传输性能
- 增加序列化/反序列化开销
- 可能导致内存使用增加

13. Mafka的客户端SDK有哪些语言支持？各有什么特点？

详细回答：

支持语言：

- Java（功能最完善，推荐使用）
- C++
- PHP
- Go
- Python
- Node.js

Java SDK特点：

- 功能最全面，支持所有高级特性
- 性能最优，经过充分优化
- 文档最完善，社区支持最好
- 支持Spring集成

其他语言SDK：

- 功能相对简单，主要支持基础的生产和消费
- 性能和稳定性不如Java版本
- 适用于特定技术栈的项目

选择建议：

- 新项目优先选择Java SDK
- 现有项目根据技术栈选择对应语言SDK
- 关键业务建议使用Java SDK



14. Mafka如何进行版本升级和兼容性管理?

详细回答:

版本策略:

- 客户端版本向后兼容
- 服务端支持多版本客户端
- 重大版本变更会提供迁移指南

升级流程:

1. 查看Release Notes了解新特性和变更
2. 在测试环境验证兼容性
3. 灰度升级部分实例
4. 全量升级

兼容性保证:

- 协议层面保持兼容
- 配置参数向后兼容
- 提供平滑迁移方案

版本选择建议:

- 生产环境使用稳定版本
- 新特性需求可选择最新版本
- 关注安全更新和bug修复

15. Mafka在美团内部的应用规模如何?

详细回答:

应用规模:

- 截止2022年末: 超过4万个主题, 280多个集群

日均用量：4万亿左右

- 覆盖业务：外卖、酒旅、金融、支付等11个BG事业部

典型应用场景：

- 外卖订单系统：订单状态变更通知
- 支付系统：支付结果异步处理
- 推荐系统：用户行为数据收集
- 监控系统：日志收集和分析

性能表现：

- 服务可用性：99.995%以上
- 消息可靠性：99.995%以上
- 发送延迟：99%控制在100ms内

次要题目

16. Mafka与开源Kafka相比有哪些优势？

详细回答：

功能增强：

- 中心化调度（Castle）
- 延迟消息/延迟队列
- Push消费模式
- 死信队列
- 消息回溯
- 事务消息

运维优化：

- 全托管服务，无需用户运维



• 完善的监控告警体系

- 自动故障转移和容灾
- 可视化管理平台

性能优化:

- 针对美团业务场景优化
- 更好的网络调度策略
- 优化的客户端实现

企业级特性:

- 多租户支持
- 权限管理
- 审计日志
- SLA保障

17. Mafka的死信队列机制是如何工作的?

详细回答:

死信队列概念:

- 处理无法正常消费的消息
- 避免消息消费卡顿影响后续消息
- 支持延迟重试机制

工作流程:

1. 消息消费失败
2. 使用SDK的retry方法将消息发送到死信队列
3. 指定延迟时间（如1分钟）
4. 继续消费后续消息

5. 延迟时间到达后，重新投递消息

配置方式：

- 在Topic管理页面开启死信功能
- 配置死信队列参数
- 在消费代码中使用retry方法

适用场景：

- 临时性消费失败（如网络抖动）
- 依赖服务暂时不可用
- 需要延迟处理的异常消息

18. Mafka如何保证消息不重复？

详细回答：

重复消息产生原因：

- 网络抖动导致重试
- 消费者重启或故障转移
- 分区重新分配

Mafka层面：

- 提供消息ID用于去重
- 支持幂等性生产者
- 消费位点管理

业务层面解决方案：

- **幂等性设计：**确保重复处理不影响结果
- **去重表：**使用数据库记录已处理消息ID
- **业务唯一键：**基于业务逻辑设计唯一标识



• **状态机：**通过状态判断是否已处理

最佳实践：

- 消费逻辑设计为幂等操作
- 使用数据库事务保证原子性
- 记录处理日志便于排查

19. Mafka的权限管理是如何实现的？

详细回答：

权限级别：

- **主题级别：**控制对特定Topic的访问
- **消费组级别：**控制消费组的创建和使用
- **操作级别：**控制生产、消费等操作权限

权限类型：

- **生产权限：**允许向Topic发送消息
- **消费权限：**允许从Topic消费消息
- **管理权限：**允许修改Topic和消费组配置

实现机制：

- 基于AppKey的身份认证
- 集成公司统一权限系统
- 支持角色和用户组管理

配置方式：

- 通过管理平台配置权限
- 支持批量权限操作
- 提供权限审计日志

20. Mafka如何进行容量规划?

详细回答:

容量评估维度:

- 消息量: 日消息数量、峰值QPS
- 消息大小: 平均消息大小、最大消息大小
- 保存时间: 消息保存天数
- 副本数量: 数据可靠性要求

计算公式:

代码块

Java



```
1  存储容量 = 日消息量 × 平均消息大小 × 保存天数 × 副本数 × 1.2 (冗余系数)
```

```
2
```

性能规划:

- 根据QPS需求确定分区数量
- 根据消费者数量规划分区数
- 考虑网络带宽和磁盘IO能力

扩容策略:

- 监控关键指标 (存储使用率、QPS等)
- 设置告警阈值
- 制定扩容预案

这些面试题目涵盖了Mafka的核心概念、技术原理、实际应用等各个方面, 按照重要程度排序, 可以帮助面试者全面了解和掌握Mafka相关知识。

⚡ 仅供内部使用，未经授权，切勿外传

